FAKE NEWS DETECTION SYSTEM

Project Documentation

Machine Learning • Natural Language Processing • TF-IDF • Linear SVM

Implemented in Google Colab using the Kaggle dataset
emineyetm/fake-news-detection-datasets

Prepared from the supplied project notebook and its recorded outputs

Project result at a glance

Item	Recorded result
Dataset after cleaning	39,103 articles
Training / testing split	31,282 / 7,821 (80% / 20%)
Best tested model	Linear Support Vector Classifier (LinearSVC)
Recorded test accuracy	99.69%
Baseline comparison	Logistic Regression: 99.21%

Academic and practical note: the reported score is from this dataset's random holdout test. It is not proof that the system can verify every real-world claim. See the evaluation and limitations sections.

Contents

1 Executive Summary

2 Introduction and Problem Statement

3 Project Objectives and Scope

4 Tools and Technologies

5 Dataset Description

6 System Design and Workflow

7 Data Preparation and NLP

8 Feature Engineering and Model Training

9 Evaluation Results

10 Prediction and Model Export

11 How to Run the Project in Google Colab

12 Limitations, Risks, and Future Improvements

13 Conclusion and References

1. Executive Summary

This project builds a supervised text-classification system that labels a news article as **Fake News (0)** or **Real News (1)**. It uses a Kaggle dataset, Python data-processing libraries, Natural Language Processing (NLP), Term Frequency–Inverse Document Frequency (TF-IDF) features, and two linear machine-learning classifiers: Logistic Regression and Linear Support Vector Classification (LinearSVC).

The notebook downloads two CSV files, combines their title and article text, removes empty and duplicate combined articles, normalizes text, splits the data into training and test sets, fits a TF-IDF vectorizer on training text, and trains both classifiers. The recorded evaluation shows 99.21% test accuracy for Logistic Regression and 99.69% for LinearSVC. LinearSVC is selected and saved with its vectorizer as a single Joblib pipeline.

The project is a useful educational demonstration of text classification. Its predictions represent learned patterns in the dataset, not independent verification of the truth of a news claim.

2. Introduction and Problem Statement

2.1 Introduction

News articles are published at high volume across digital platforms. Machine-learning methods can help screen article text for patterns associated with previously labelled examples. NLP transforms unstructured language into representations that a statistical classifier can process.

2.2 Problem statement

The project addresses a binary classification problem: given a news headline and article body, predict whether the example resembles the dataset's fake-news class or real-news class.

2.3 Intended use

The system is intended for learning, demonstration, and preliminary text screening. It should not be used as the sole basis for publishing corrections, accusing a source of misinformation, making high-impact decisions, or declaring a claim true or false.

3. Project Objectives and Scope

- Download and inspect the Kaggle dataset from Google Colab.
- Combine news titles and article bodies into a single text feature.
- Prepare clean, labelled data for supervised learning.
- Convert text into TF-IDF features with unigrams and bigrams.
- Train and compare Logistic Regression and LinearSVC.
- Evaluate results using accuracy, precision, recall, F1-score, and a confusion matrix.
- Save the best model and vectorizer together for later inference.
- Provide a function to classify new user-entered news text.

Out of scope: automated source verification, web-based fact checking, evidence retrieval, multilingual support, live news monitoring, and human editorial review are not implemented in the supplied notebook.

4. Tools and Technologies

Technology	Purpose in this project
Python / Google Colab	Execution environment and notebook-based workflow.
KaggleHub	Download the selected Kaggle dataset.
Pandas / NumPy	Load, clean, combine, and inspect tabular data.
Regular expressions (re)	Normalize HTML tags, URLs, email addresses, and whitespace.
scikit-learn	Train/test split, TF-IDF, classifiers, metrics, and pipeline.
Matplotlib / Seaborn	Visualize class distribution, model comparison, and confusion matrix.
Joblib	Serialize the trained pipeline for later reuse.

5. Dataset Description

The notebook downloads the dataset with the KaggleHub identifier **emineyetm/fake-news-detection-datasets**. It finds two CSV files inside the downloaded directory: **Fake.csv** and **True.csv**.

Dataset item	Observed value in notebook
CSV files	Fake.csv and True.csv
Rows after initial concatenation	44,898
Columns	title, text, subject, date, label
Initial fake label	0
Initial real label	1
Missing values at initial inspection	0 in each listed column
Rows after removing empty combined text and duplicate articles	39,103
Cleaned fake examples	17,907
Cleaned real examples	21,196

Label interpretation

- **0 = Fake News:** examples read from Fake.csv.
- **1 = Real News:** examples read from True.csv.

These labels are inherited from the dataset's file organization. The notebook does not independently fact-check the content or verify the editorial reliability of each article.

Data quality consideration

The notebook reports 44,898 rows before cleaning and 39,103 afterward. This is a reduction of 5,795 rows from removing records whose combined title/body is empty and duplicate combined text. The notebook's shown code removes duplicates based on combined text, not on title alone.

6. System Design and Workflow

The implemented processing pipeline follows these stages:

Stage	Operation	Output
1. Download	KaggleHub retrieves dataset files.	Local dataset directory
2. Load	Read Fake.csv and True.csv; assign labels.	Combined DataFrame
3. Clean records	Normalize column names, fill text nulls, combine title/body, drop empty/duplicate text.	39,103 labelled rows
4. Normalize text	Lowercase; replace URLs and emails with tokens; remove HTML tags and repeated whitespace.	Prepared text
5. Split	Stratified 80/20 split with random_state=42.	31,282 train / 7,821 test

Stage	Operation	Output
6. Vectorize	Fit TF-IDF on train text and transform test text.	Sparse numerical matrices
7. Train	Fit Logistic Regression and LinearSVC.	Two classifiers
8. Evaluate	Calculate accuracy, classification report, and confusion matrix.	Comparison metrics
9. Export	Package TF-IDF and selected classifier in a Pipeline and save with Joblib.	fake_news_detection_model.joblib
10. Infer	Load pipeline and classify new headline/article text.	Predicted class label

7. Data Preparation and NLP

7.1 Combining headline and body

The code creates a field named **news** by joining the title and article body with a space. If the title column is absent, it creates an empty title field. Missing title and body values are replaced with empty strings before concatenation.

7.2 Record filtering

Rows with empty combined text are removed. Duplicate records are removed based on the combined news text. Only rows with labels 0 or 1 are retained, and labels are cast to integers.

7.3 Text normalization

- Convert text to lowercase.
- Remove HTML tags using a regular expression.
- Replace URLs with the token URL.
- Replace email addresses with the token EMAIL.
- Collapse repeated whitespace and trim leading/trailing spaces.

The notebook does not stem words or apply lemmatization. This is a reasonable baseline because TF-IDF can represent terms directly, but alternative preprocessing choices could be tested in future experiments.

7.4 Train/test separation

The data is split with test_size=0.20, random_state=42, and stratify=y. The resulting training set has 31,282 records and the test set has 7,821 records. Stratification preserves approximately the same class proportions in each split.

8. Feature Engineering and Model Training

8.1 TF-IDF settings

Parameter	Configured value	Meaning
stop_words	english	Exclude common English stop words.
max_features	100,000	Cap the vocabulary at 100,000 features.
ngram_range	(1, 2)	Use single words and adjacent two-word phrases.
min_df	2	Ignore terms appearing in fewer than two documents.
max_df	0.95	Ignore terms appearing in more than 95% of documents.
sublinear_tf	True	Apply sublinear term-frequency scaling.
dtype	float32	Use 32-bit floating-point values for features.

The TF-IDF vectorizer is fitted only on X_train, then used to transform X_test. This avoids fitting the vocabulary and inverse-document-frequency statistics on the held-out test set.

8.2 Logistic Regression

The baseline classifier is LogisticRegression(max_iter=1000, C=2.0, random_state=42). It learns a linear decision boundary in the TF-IDF feature space.

8.3 Linear Support Vector Classifier

The second classifier is LinearSVC(C=1.0, random_state=42, max_iter=5000). It learns a linear separating hyperplane and is often effective for sparse text representations.

The notebook selects the model with the higher test accuracy. The selected model and the already-fitted TF-IDF vectorizer are combined in a scikit-learn Pipeline before saving.

9. Evaluation Results

The following numbers are copied from the recorded output of the supplied notebook. They are not estimates or newly generated results.

Metric	Logistic Regression	LinearSVC
Test accuracy	99.2073% (displayed as 99.21%)	99.6931% (displayed as 99.69%)
Test examples	7,821	7,821
Fake class support	3,582	3,582
Real class support	4,239	4,239
Selected by notebook?	No	Yes

Classification report (LinearSVC)

Class / metric	Precision	Recall	F1-score	Support
Fake News	1.00 (rounded)	1.00 (rounded)	1.00 (rounded)	3,582
Real News	1.00 (rounded)	1.00 (rounded)	1.00 (rounded)	4,239
Accuracy	—	—	99.69%	7,821
Macro average	1.00 (rounded)	1.00 (rounded)	1.00 (rounded)	7,821
Weighted average	1.00 (rounded)	1.00 (rounded)	1.00 (rounded)	7,821

The classification report rounds precision, recall, and F1-score to two decimal places, so the displayed 1.00 values should not be interpreted as proof of mathematically perfect performance. The more precise recorded LinearSVC accuracy is 99.693134%.

Interpretation

- LinearSVC outperformed Logistic Regression by approximately 0.49 percentage points on this test split.
- The model appears to distinguish examples in this dataset very well.
- A random holdout from the same dataset may not reflect performance on new publishers, later news events, other languages, or deliberately manipulated examples.
- Near-perfect scores can occur when datasets contain source, topic, writing-style, repeated-content, or collection artifacts that correlate with the labels.

10. Prediction and Model Export

10.1 Saved artifact

The notebook saves the selected TF-IDF-plus-classifier pipeline to **/content/fake_news_detection_model.joblib**. It then reloads the saved file and uses the loaded pipeline to predict labels for new text.

10.2 Inference function

```
def predict_news(title, article_text):
combined_news = str(title) + " " + str(article_text)
combined_news = clean_text(combined_news)

if not combined_news.strip():
raise ValueError("Please provide a news headline or article text.")

prediction = loaded_model.predict([combined_news])[0]
return "FAKE NEWS" if prediction == 0 else "REAL NEWS"
```

The function joins the supplied title and article body, applies the same text-normalization function, and passes the result to the loaded pipeline. The output is a binary label. The current pipeline does not return a calibrated probability or an evidence-based explanation.

10.3 Downloading the model

The notebook uses **google.colab.files.download** to download the Joblib file. The model file should be kept together with compatible Python and scikit-learn versions. Only load Joblib files from trusted sources, because serialized Python objects can execute code when loaded.

10.4 Prediction output example

The supplied notebook uses placeholder text (“Example news headline” and generic article text) and the recorded prediction is FAKE NEWS. This is only a demonstration of the code path; it is not a meaningful validation example and should not be reported as evidence that the placeholder is false.

11. How to Run the Project in Google Colab

1. Open Google Colab and create a new notebook.
2. Run the installation cell for kagglehub, pandas, numpy, scikit-learn, matplotlib, seaborn, and joblib.
3. Run the imports and set the random seed to 42.
4. Download the dataset using kagglehub.dataset_download('emineyetm/fake-news-detection-datasets').
5. Find the CSV files recursively and load Fake.csv and True.csv.
6. Run the cleaning and label preparation cells; check that the cleaned count is consistent with 39,103 for the supplied notebook run.
7. Create X and y, then split the data with stratify=y and test_size=0.20.
8. Fit TF-IDF on the training text only and transform the test text.
9. Train Logistic Regression and LinearSVC; compare their metrics.
10. Save the selected pipeline with joblib.dump and test it using real headline/body examples.
11. Download the model and prediction CSV if you need local copies.

Reproducibility note: the dataset version, library versions, and runtime environment can change. The recorded numbers describe the uploaded notebook execution; a fresh run may produce slightly different results.

12. Limitations, Risks, and Future Improvements

12.1 Current limitations

- **Dataset dependence:** the classifier learns the labels and patterns of this dataset, not universal truth.
- **Possible source and topic bias:** wording, publisher identity, political topic, and article style may correlate with labels.
- **Random split limitation:** related stories or similar articles may appear in both training and test partitions; the notebook does not perform a publisher-held-out or time-based evaluation.
- **No evidence verification:** the model does not retrieve primary sources, cross-check claims, or inspect citations.
- **Binary output:** the model only returns Fake News or Real News, with no uncertainty threshold or review category.
- **Language scope:** the preprocessing and stop-word configuration are designed for English text.
- **Demo input:** the placeholder example should not be treated as an actual fact-check.

12.2 Recommended improvements

- Deduplicate and group near-identical articles before splitting to reduce leakage risk.
- Add a time-based test set and, where possible, a publisher/source-held-out test set.
- Evaluate on an independent, recent dataset that was not used for model development.
- Inspect false positives and false negatives and report the full confusion matrix counts.
- Use cross-validation and systematic hyperparameter tuning without repeatedly optimizing against the test set.
- Add a third outcome such as “Uncertain — manual fact-check required,” based on a validated decision policy.
- Build a user interface with clear warnings, source links, and a human review step.
- Experiment with transformer models such as BERT only after establishing reliable baselines and evaluation controls.
- Document dataset license, provenance, collection date, and any permitted-use restrictions before deployment.

Ethical note: a wrong “fake” label can damage a publisher's or person's reputation, while a wrong “real” label can amplify misinformation. Human review and independent evidence are essential for consequential use.

13. Conclusion

The supplied notebook implements an end-to-end educational fake-news text-classification workflow: dataset download, data inspection, cleaning, TF-IDF feature extraction, model comparison, evaluation, and model export. On the recorded 80/20 split of 39,103 cleaned articles, LinearSVC achieved 99.69% accuracy and was selected over Logistic Regression at 99.21%.

The result is promising for this dataset, but it should be described as classification performance on a held-out portion of the same collection—not as proof that the system can reliably determine truth in the open world. The strongest next step is a more rigorous evaluation using time-separated and source-separated data, followed by human-reviewed fact-checking support.

14. References

- Kaggle dataset identifier used by the notebook: emineyetm/fake-news-detection-datasets.
- scikit-learn documentation — TfidfVectorizer: https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html
- scikit-learn documentation — LogisticRegression: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html
- scikit-learn documentation — LinearSVC: https://scikit-learn.org/stable/modules/generated/sklearn.svm.LinearSVC.html
- scikit-learn documentation — classification metrics: https://scikit-learn.org/stable/modules/model_evaluation.html
- KaggleHub documentation / package: https://github.com/Kaggle/kagglehub
- Project implementation source: user-provided Google Colab notebook, Untitled36.ipynb.

End of documentation • Fake News Detection System • Prepared from the uploaded notebook and recorded outputs.